

FIȘĂ DE LUCRU

Subprograme (funcții) în Python

Clasa a IX-a – Informatică (CS) – Matematica-Informatică

1. Noțiuni teoretice

Ce este un subprogram?

Un subprogram (funcție) este un bloc de cod reutilizabil care realizează o sarcină specifică. Funcțiile permit organizarea modulară a codului și evitarea repetării aceluiași cod.

Sintaxa de definire în Python:

```
def nume_funcție(param1, param2, ...):  
    # corpul funcției (instrucțiuni indentate)  
    return rezultat
```

Componentele unei funcții:

def – cuvânt cheie obligatoriu care marchează începutul definiției funcției

nume_funcție – numele ales de programator (se recomandă snake_case)

parametri formali – variabilele din paranteze care primesc valori la apel (opționali)

corpul funcției – instrucțiunile indentate care se execută la apel

return – instrucțiunea care returnează rezultatul (opțională)

Parametri formali vs. parametri efectivi:

Parametrii formali sunt variabilele declarate în definiția funcției. Parametrii efectivi (argumente) sunt valorile transmise la apelul funcției.

```
def suma(a, b):          # a, b = parametri formali  
    return a + b  
  
rez = suma(5, 3)        # 5, 3 = parametri efectivi (argumente)
```

Variabile locale vs. globale:

Variabilele definite în interiorul unei funcții sunt locale – există doar în funcție. Variabilele definite în afara funcțiilor sunt globale – accesibile din orice punct al programului.

2. Exemple rezolvate pas cu pas

Exemplul 1: Funcție care calculează pătratul unui număr

```
def patrat(n):  
    return n * n
```

```
x = int(input('Introdu un numar: '))  
print('Pătratul este:', patrat(x))
```

Pas 1: Se definește funcția patrat cu parametrul formal n.

*Pas 2: Corpul funcției calculează $n*n$ și returnează rezultatul.*

Pas 3: Se citește un număr de la tastatură și se apelează funcția cu acel număr ca parametru efectiv.

Exemplul 2: Funcție care verifică dacă un număr este prim

```
def este_prim(n):
```

```

    if n < 2:
        return False
    for d in range(2, int(n**0.5) + 1):
        if n % d == 0:
            return False
    return True

nr = int(input('Introdu un numar: '))
if este_prim(nr):
    print(nr, 'este prim')
else:
    print(nr, 'nu este prim')

```

Pas 1: Funcția primește un număr n ca parametru.

Pas 2: Dacă $n < 2$, returnează False (nu este prim).

Pas 3: Se parcurg posibili divizori de la 2 până la radical din n. Dacă se găsește un divizor, returnează False.

Pas 4: Dacă nu s-a găsit niciun divizor, returnează True.

Exemplul 3: Funcție cu mai mulți parametri – calculul mediei a 3 note

```

def media(n1, n2, n3):
    return (n1 + n2 + n3) / 3

nota1 = float(input('Nota 1: '))
nota2 = float(input('Nota 2: '))
nota3 = float(input('Nota 3: '))
m = media(nota1, nota2, nota3)
print('Media este:', round(m, 2))

```

Observați utilizarea funcției predefinite round() pentru rotunjire la 2 zecimale.

3. Exerciții propuse

Exercițiul 1 (ușor)

Scrieți o funcție `dublu(n)` care returnează dublul numărului primit ca parametru. Apelați funcția pentru valorile 5, 12 și 0.

Exercițiul 2 (ușor)

Scrieți o funcție `salut(ume)` care afișează mesajul: Bun venit, [nume]! Apelați funcția pentru numele a 3 colegi.

Exercițiul 3 (mediu)

Scrieți o funcție `celsius_fahrenheit(c)` care convertește temperatura din Celsius în Fahrenheit ($F = C * 9/5 + 32$). Afișați temperatura în Fahrenheit pentru: 0, 20, 37 și 100 grade Celsius.

Exercițiul 4 (mediu)

Scrieți o funcție `suma_cifre(n)` care returnează suma cifrelor unui număr natural. De exemplu, `suma_cifre(1234)` returnează 10.

Exercițiul 5 (avansat)

Scrieți două funcții: `cmmdc(a, b)` care calculează cel mai mare divizor comun folosind algoritmul lui Euclid, și `cmmmc(a, b)` care calculează cel mai mic multiplu comun folosind formula: $cmmmc = a*b / cmmdc(a,b)$. Testați pentru perechile (12, 8), (15, 25) și (7, 3).

Exercițiul 6 (avansat)

Scrieți o funcție `nr_div(n)` care returnează numărul de divizori ai unui număr natural n. Apoi, folosiți această funcție pentru a afișa toate numerele din intervalul [1, 100] care au exact 6 divizori.

4. Erori frecvente și cum le evităm

Eroare 1: Lipsa instrucțiunii return

```
# GRESIT - functia nu returneaza nimic (returneaza None)
def suma(a, b):
    a + b

# CORECT
def suma(a, b):
    return a + b
```

Eroare 2: Indentare greșită

```
# GRESIT - lipseste indentarea
def salut():
print('Bun venit!')

# CORECT
def salut():
    print('Bun venit!')
```

Eroare 3: Apelul funcției înainte de definiție

```
# GRESIT - functia nu este inca definita
```

```
rezultat = patrat(5)
def patrat(n):
    return n * n

# CORECT - mai intai definim, apoi apelam
def patrat(n):
    return n * n
rezultat = patrat(5)
```

5. Rezumat

- Funcțiile se definesc cu **def** urmat de numele funcției și parametri între paranteze.
- Instrucțiunea **return** returnează rezultatul funcției către apelant.
- Parametrii formali primesc valorile parametrilor efectivii la momentul apelului.
- Variabilele locale există doar în interiorul funcției; variabilele globale sunt accesibile de oriunde.
- Funcțiile predefinite (abs, round, len, min, max, sum etc.) sunt subprograme deja scrise în Python.